

图片颜色可视化分析工具 - 项目说明文档

1. 项目概述

图片颜色可视化分析工具是一个基于Web技术的应用，旨在帮助用户分析图片中的颜色分布，评估颜色和谐度，并通过多种图表形式直观展示分析结果。该工具支持两种版本：现代化的2.0版本和轻量化的1.0版本，满足不同用户的需求。

1.1 项目目标

- 提供直观的图片颜色分析功能
- 支持多种图表类型展示颜色分布
- 评估图片颜色的和谐度
- 提供历史记录和结果导出功能
- 适配不同设备的响应式设计



2.1 核心功能

- 图片上传**: 支持点击上传和拖拽上传，支持JPG、PNG、WebP格式
- 颜色聚类分析**: 使用K-Means算法对图片颜色进行聚类，可调节聚类数量K值
- 多种图表类型**: 支持饼图、柱状图、热力图、雷达图四种图表类型
- 颜色空间选择**: 支持RGB和LAB两种颜色空间
- 颜色和谐度评估**: 支持通过DeepSeek API或模拟评估颜色和谐度
- 历史记录**: 保存最近3次分析结果，支持快速查看历史分析

- **结果导出**：支持导出为JSON和CSV格式

2.2 版本特性

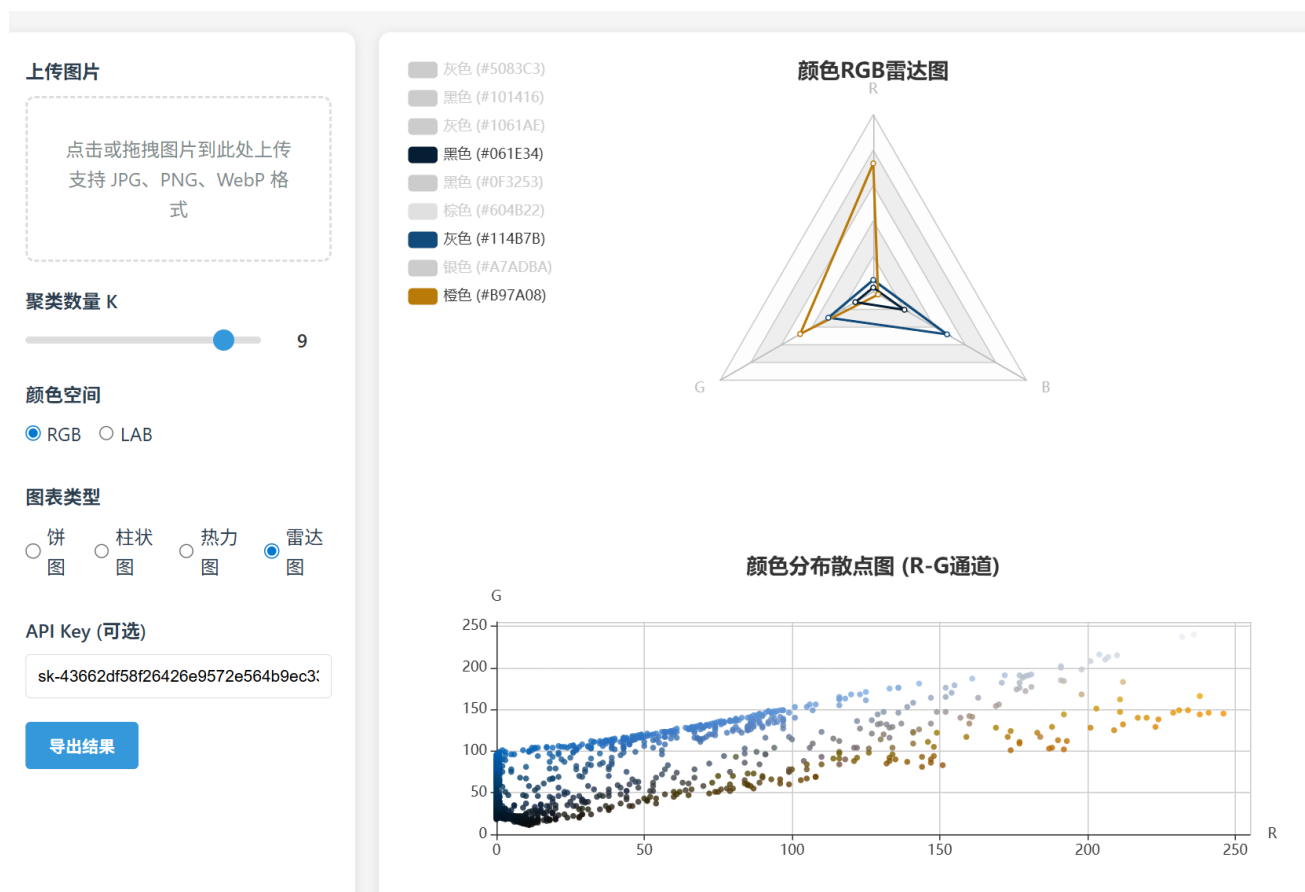
版本2.0

- 星空背景装饰
- 玻璃态设计
- 流星动画效果
- 现代化界面



版本1.0

- 简洁界面
- 更快的加载速度
- 轻量化设计
- 核心功能完整



3. 技术实现

3.1 技术栈

- **前端框架**: 原生HTML5 + CSS3 + JavaScript
- **图表库**: ECharts 5.x
- **颜色空间转换**: 实现了RGB到LAB颜色空间的转换
- **聚类算法**: 实现了K-Means聚类算法
- **API集成**: 集成DeepSeek API进行颜色和谐度评估
- **响应式设计**: 适配不同屏幕尺寸

3.2 核心模块

3.2.1 图片处理模块

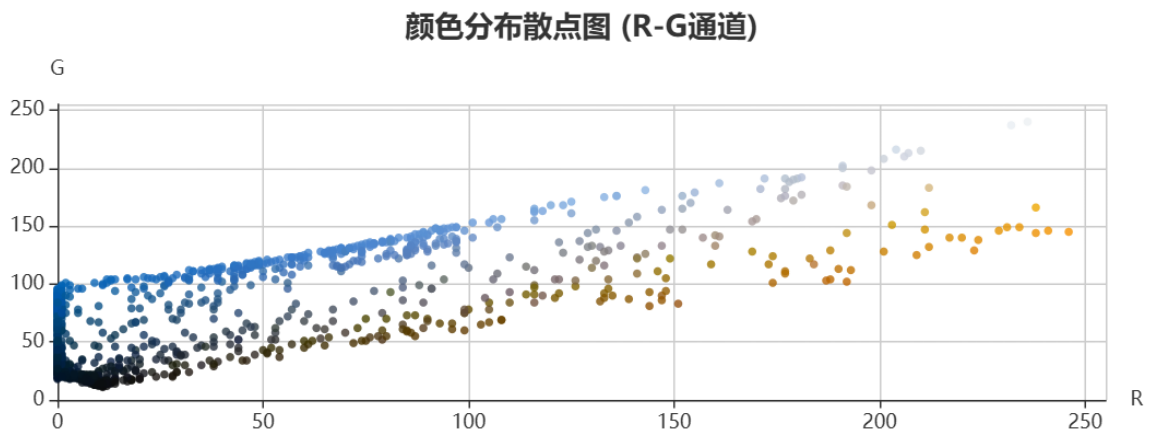
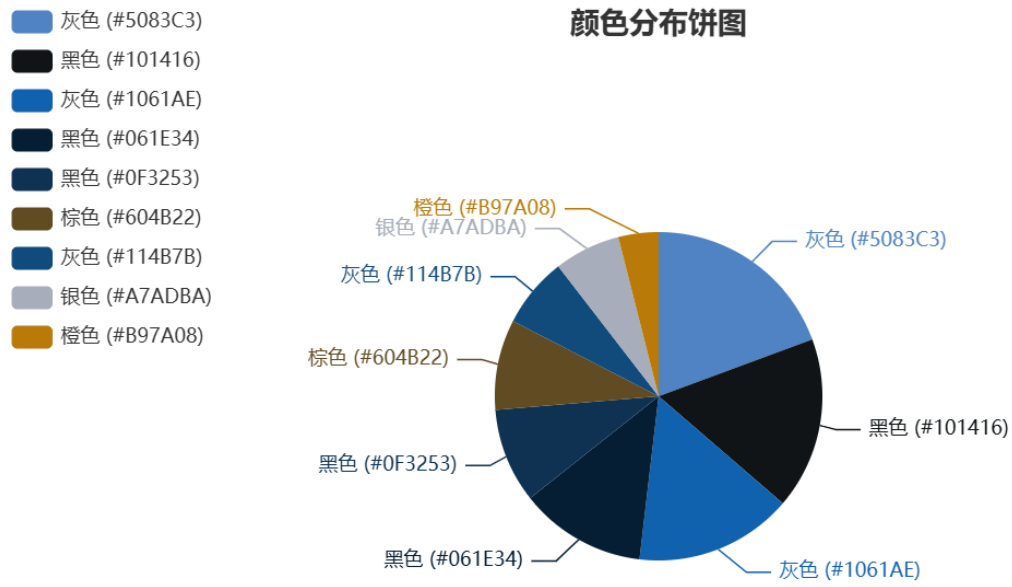
- 负责图片上传和预览
- 图片缩放处理 (最大尺寸400x400)
- 像素数据提取和处理

3.2.2 聚类分析模块

- K-Means聚类算法实现
- 支持RGB和LAB颜色空间
- 聚类结果排序和统计

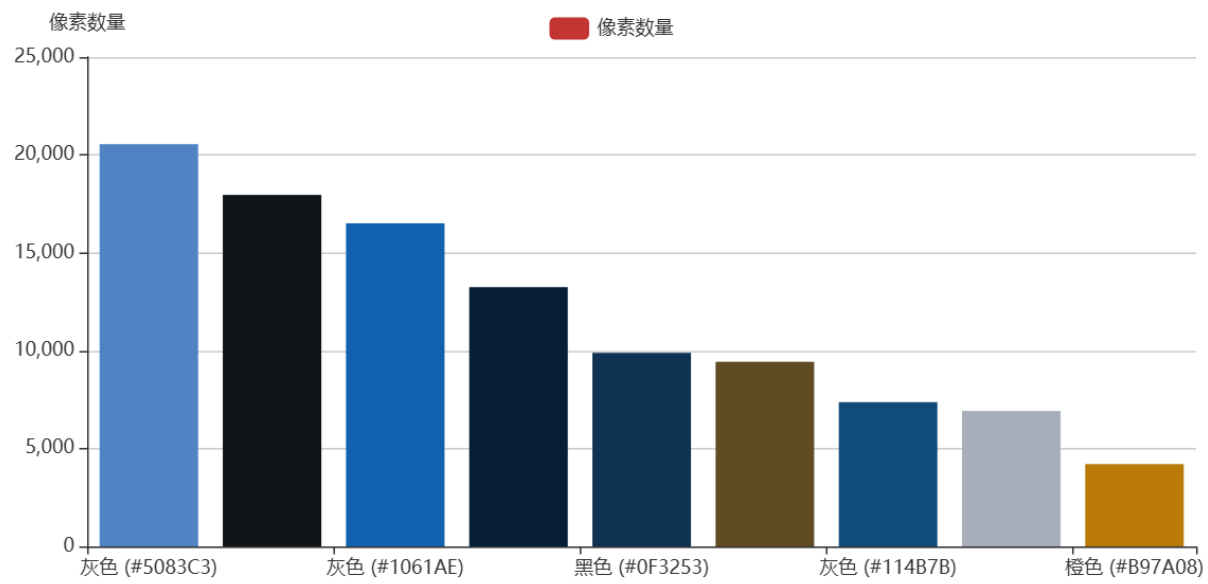
3.2.3 图表展示模块

- 饼图&散点图：展示颜色分布比例、分布散点（其中散点图会始终显示在下方）



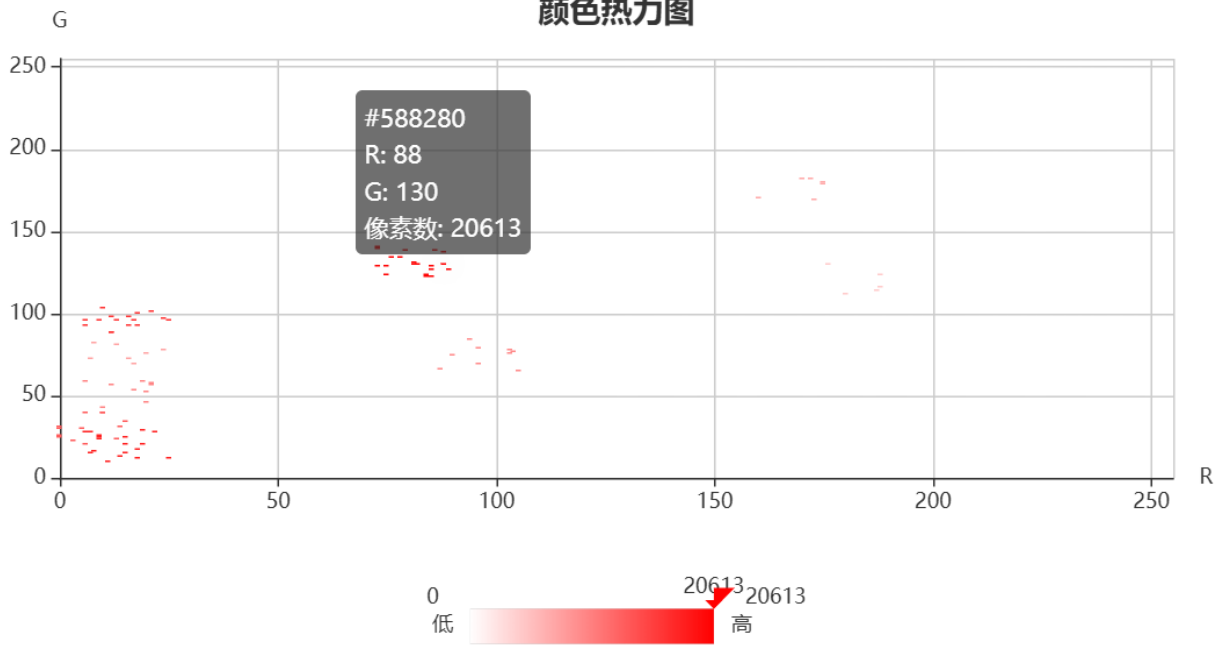
- 柱状图：展示颜色像素数量

颜色分布柱状图



- 热力图：展示颜色分布密度

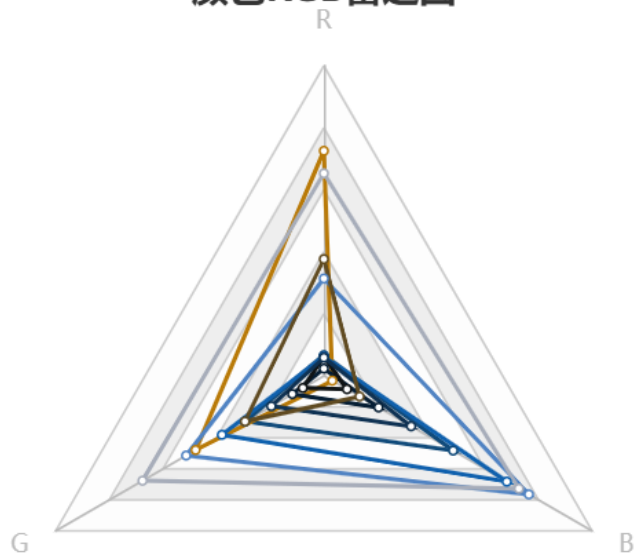
颜色热力图



- 雷达图：展示颜色RGB值

- 灰色 (#5083C3)
- 黑色 (#101416)
- 灰色 (#1061AE)
- 黑色 (#061E34)
- 黑色 (#0F3253)
- 棕色 (#604B22)
- 灰色 (#114B7B)
- 银色 (#A7ADBA)
- 橙色 (#B97A08)

颜色RGB雷达图



3.2.4 和谐度评估模块

- 支持DeepSeek API评估
- 模拟评估模式（当API不可用时会有弹窗显示）
- 评估结果展示

ilovedianaforever.github.io 显示

API Key 不可用或已过期，请检查您的API Key是否正确。

系统将自动切换到模拟评估模式。

确定

颜色和谐度评估

和谐度评分: 7/10

深蓝与金色形成优雅对比，整体沉稳专业，但部分中间色调略显冗余，可精简以提升和谐度。

历史记录



K: 8
RGB
2026/4/13 15:17:45



K: 7
RGB
2026/4/13 15:17:06



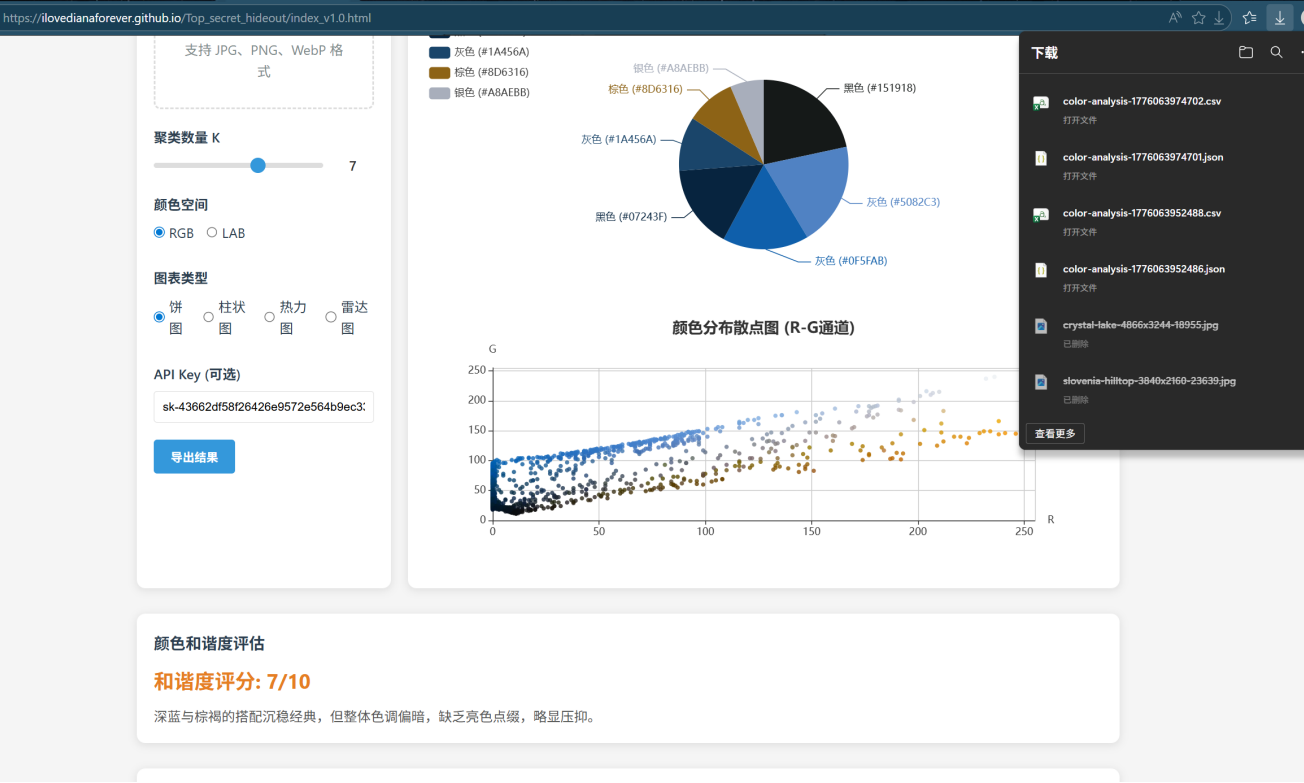
K: 9
RGB
2026/4/13 15:08:25

3.2.5 历史记录模块

- 保存最近3次分析结果（并非永久储存，用户刷新网页/关闭网页再打开会消失）
- 历史记录预览和加载
- 缩略图生成

3.2.6 导出模块

- JSON格式导出
- CSV格式导出



> index_v1.0_copy.html	<> index_v2.0.html	<> index_v1.0.html	<> index.html	更新日志	color-analysis-1776063952488.csv x
导出结果 > color-analysis-1776063952488.csv > data					
1	颜色, 十六进制, 名称, 像素数, 占比				
2	11, 30, 47, #0B1E2F, 黑色, 40274, 37.85%				
3	22, 95, 165, #165FA5, 灰色, 27229, 25.59%				
4	110, 146, 194, #6E92C2, 灰色, 23291, 21.89%				
5	87, 71, 39, #574727, 棕色, 10882, 10.23%				
6	180, 120, 10, #B4780A, 棕色, 4724, 4.44%				

index_v1.0_copy.html <> index_v2.0.html <> index_v1.0.html <> index.html 更新日志 color-analysis-1776063952486.json x

输出结果 > {} color-analysis-1776063952486.json > {} clusters > {} 1

```
5  "clusters": [  
6    {  
7      "color": [  
8        11,  
9        30,  
10       47  
11     ],  
12     "hex": "#0B1E2F",  
13     "name": "黑色",  
14     "count": 40274,  
15     "percentage": 37.8515037593985  
16   },  
17   {  
18     "color": [  
19       22,  
20       95,  
21       165  
22     ],  
23     "hex": "#165FA5",  
24     "name": "灰色",  
25     "count": 27229,  
26     "percentage": 25.591165413533833  
27   },  
28   {  
29     "color": [  
30       110,  
31       146,  
32       194  
33     ],  
34     "hex": "#6E92C2",  
35     "name": "灰色",
```

4. 代码结构

4.1 文件结构

文件

测试用图片

- crystal-lake-4866x...
- slovenia-hilltop-38...

导出结果

- color-analysis-177...
- color-analysis-177...
- color-analysis-177...
- color-analysis-177...

演示截图

- 8bb08726-c875-4f...
- 8f901a99-4e2e-43...
- 2206e6e4-e339-44...
- 3878a823-7bc6-4e...
- 06123b42-57c6-49...
- 67653536-b129-4...
- 74701493-9b53-4...
- eb0367b5-e9e2-4...
- edf9e3ee-5f35-4a...

说明文档.docx

项目说明文档.md

JS d3.v3.min.js

JS dataTool.min.js

JS echarts-gl.min.js

JS echarts.min.js

JS ecStat.min.js

< > index_v1.0_copy.html

< > index_v1.0.html

< > index_v2.0_copy.html

```
• <> index_v2.0.html
• <> index.html
```

```
4.3-4.19_big_homework/
├─ 演示截图/           # 项目演示截图
├─ d3.v3.min.js         # D3.js库
├─ dataTool.min.js      # 数据处理工具
├─ ecStat.min.js        # ECharts统计工具
├─ echarts-gl.min.js    # ECharts GL扩展
├─ echarts.min.js       # ECharts核心库
├─ index.html           # 版本选择页面
├─ index_v1.0.html      # 版本1.0实现
├─ index_v1.0_copy.html # 版本1.0副本
├─ index_v2.0.html      # 版本2.0实现
├─ index_v2.0_copy.html # 版本2.0副本
└─ 说明文档.docx       # 原说明文档
```

4.2 核心代码文件

4.2.1 index.html

版本选择页面，提供两个版本的入口链接。

4.2.2 index_v2.0.html

版本2.0的完整实现，包含现代化的星空背景和玻璃态设计。

4.2.3 index_v1.0.html

版本1.0的完整实现，采用简洁的界面设计。

5. 核心算法

5.1 K-Means聚类算法

```
function kmeans(pixels, k, colorSpace) {
  // 转换颜色空间
  const transformedPixels = pixels.map(pixel => {
    if (colorSpace === 'lab') {
      return rgbToLab(pixel[0], pixel[1], pixel[2]);
    }
    return pixel;
  });

  // 随机初始化中心点（从样本中选择）
  let centroids = [];
  const usedIndices = new Set();

  for (let i = 0; i < k; i++) {
```

```

    let index;
    do {
        index = Math.floor(Math.random() * transformedPixels.length);
    } while (usedIndices.has(index));
    usedIndices.add(index);
    centroids.push([...transformedPixels[index]]);
}

// 迭代聚类过程
let clusters = new Array(k).fill(0).map(() => []);
let maxIterations = 50;
let convergenceThreshold = 1e-4;
let iteration = 0;

while (iteration < maxIterations) {
    // 分配像素到最近的中心点
    clusters = new Array(k).fill(0).map(() => []);
    for (let i = 0; i < transformedPixels.length; i++) {
        let minDistance = Infinity;
        let closestCentroid = 0;

        for (let j = 0; j < k; j++) {
            const distance = calculateDistance(transformedPixels[i], centroids[j]);
            if (distance < minDistance) {
                minDistance = distance;
                closestCentroid = j;
            }
        }

        clusters[closestCentroid].push(i);
    }

    // 计算新的中心点
    let newCentroids = [];
    let hasEmptyCluster = false;

    for (let i = 0; i < k; i++) {
        if (clusters[i].length === 0) {
            // 处理空聚类, 重新随机选择中心点
            let index;
            do {
                index = Math.floor(Math.random() * transformedPixels.length);
            } while (usedIndices.has(index));
            usedIndices.add(index);
            newCentroids.push([...transformedPixels[index]]);
            hasEmptyCluster = true;
        } else {
            // 计算平均值
            const centroid = [0, 0, 0];
            for (const index of clusters[i]) {
                for (let j = 0; j < 3; j++) {
                    centroid[j] += transformedPixels[index][j];
                }
            }

```

```

    }
    for (let j = 0; j < 3; j++) {
        centroid[j] /= clusters[i].length;
    }
    newCentroids.push(centroid);
}
}

// 检查收敛
let maxChange = 0;
for (let i = 0; i < k; i++) {
    const change = calculateDistance(centroids[i], newCentroids[i]);
    if (change > maxChange) {
        maxChange = change;
    }
}

centroids = newCentroids;
iteration++;

if (!hasEmptyCluster && maxChange < convergenceThreshold) {
    break;
}
}

// 转换回RGB颜色空间 (如果使用的是LAB)
const result = [];
for (let i = 0; i < k; i++) {
    let color;
    if (colorSpace === 'lab') {
        // 简化处理, 直接使用原始像素的平均值
        const rgbSum = [0, 0, 0];
        for (const index of clusters[i]) {
            for (let j = 0; j < 3; j++) {
                rgbSum[j] += pixels[index][j];
            }
        }
        color = rgbSum.map(val => Math.round(val / clusters[i].length));
    } else {
        color = centroids[i].map(val => Math.round(val));
    }

    result.push({
        color: color,
        count: clusters[i].length,
        percentage: (clusters[i].length / pixels.length) * 100
    });
}

// 按像素数量排序
result.sort((a, b) => b.count - a.count);
return result;
}

```

5.2 RGB到LAB颜色空间转换

```
function rgbToLab(r, g, b) {  
    // RGB到XYZ  
    r /= 255;  
    g /= 255;  
    b /= 255;  
  
    r = r > 0.04045 ? Math.pow((r + 0.055) / 1.055, 2.4) : r / 12.92;  
    g = g > 0.04045 ? Math.pow((g + 0.055) / 1.055, 2.4) : g / 12.92;  
    b = b > 0.04045 ? Math.pow((b + 0.055) / 1.055, 2.4) : b / 12.92;  
  
    r *= 100;  
    g *= 100;  
    b *= 100;  
  
    const x = r * 0.4124 + g * 0.3576 + b * 0.1805;  
    const y = r * 0.2126 + g * 0.7152 + b * 0.0722;  
    const z = r * 0.0193 + g * 0.1192 + b * 0.9505;  
  
    // XYZ到LAB  
    const xRef = 95.047;  
    const yRef = 100.000;  
    const zRef = 108.883;  
  
    let xRatio = x / xRef;  
    let yRatio = y / yRef;  
    let zRatio = z / zRef;  
  
    xRatio = xRatio > 0.008856 ? Math.pow(xRatio, 1/3) : (7.787 * xRatio) + (16/116);  
    yRatio = yRatio > 0.008856 ? Math.pow(yRatio, 1/3) : (7.787 * yRatio) + (16/116);  
    zRatio = zRatio > 0.008856 ? Math.pow(zRatio, 1/3) : (7.787 * zRatio) + (16/116);  
  
    const L = (116 * yRatio) - 16;  
    const a = 500 * (xRatio - yRatio);  
    const labB = 200 * (yRatio - zRatio);  
  
    return [L, a, labB];  
}
```

6. 使用指南

6.1 基本使用流程

1. **选择版本**：访问 `index.html` 选择喜欢的版本（2.0或1.0）
2. **上传图片**：点击或拖拽图片到上传区域
3. **调整参数**：
 - 调整聚类数量K值（2-10）

- 选择颜色空间 (RGB或LAB)
- 选择图表类型 (饼图、柱状图、热力图、雷达图)

4. 查看结果:

- 查看颜色分布图表
- 查看颜色和谐度评估
- 查看历史记录

5. 导出结果: 点击"导出结果"按钮, 导出为JSON和CSV格式

6.2 高级功能

- API Key设置:** 在API Key输入框中输入DeepSeek API Key, 获得更准确的颜色和谐度评估
- 历史记录:** 点击历史记录中的缩略图, 快速加载之前的分析结果
- 响应式设计:** 在不同设备上都能获得良好的使用体验

7. 版本差异

特性	版本1.0	版本2.0
界面设计	简洁风格	现代化风格
背景效果	无	星空背景 + 流星动画
视觉效果	普通卡片	玻璃态设计
加载速度	更快	稍慢 (由于动画效果)
功能完整性	完整	完整
适用场景	性能优先	视觉体验优先

8. 性能优化

- 图片处理优化:** 对大图片进行缩放处理, 限制最大尺寸为400x400
- 聚类算法优化:** 使用防抖函数减少频繁聚类计算
- 散点图优化:** 随机采样1000个点, 提高渲染性能
- 渲染优化:** 使用requestAnimationFrame和setTimeout避免UI阻塞

9. 未来优化方向

- 支持更多图片格式:** 如SVG、GIF等
- 增加更多图表类型:** 如3D饼图、词云等
- 增强颜色分析功能:** 如颜色趋势分析、颜色搭配建议等
- 添加批量分析功能:** 支持多张图片同时分析
- 增加用户账户系统:** 保存分析历史到云端
- 优化移动端体验:** 进一步提升移动端的交互体验
- 增加更多AI评估维度:** 如颜色情感分析、品牌色彩匹配等

10. 技术亮点

- 双版本设计**：同时提供现代化和轻量化两个版本，满足不同用户需求
- 多种图表类型**：支持四种不同类型的图表，从不同角度展示颜色分布
- 颜色空间转换**：实现了RGB到LAB颜色空间的转换，提供更准确的颜色分析
- K-Means聚类算法**：自行实现了K-Means聚类算法，支持自定义聚类数量
- AI集成**：集成DeepSeek API进行颜色和谐度评估
- 响应式设计**：适配不同屏幕尺寸，提供良好的跨设备体验
- 性能优化**：通过多种手段优化性能，确保流畅的用户体验
- 历史记录功能**：保存最近的分析结果，方便用户快速查看
- 结果导出**：支持导出为JSON和CSV格式，方便进一步分析
- 现代化UI设计**：版本2.0采用星空背景和玻璃态设计，提供出色的视觉体验

11. 项目总结

图片颜色可视化分析工具是一个功能完整、界面美观的Web应用，通过直观的方式帮助用户分析图片中的颜色分布和谐度。该项目展示了如何使用前端技术实现复杂的颜色分析功能，包括K-Means聚类算法、颜色空间转换、多种图表展示等。目前已通过Github Pages设置为public网站，所有人都可以来访问，我的个人github主页也已在页面最下方给出。

项目的双版本设计体现了对不同用户需求的考虑，版本2.0提供了现代化的视觉体验，版本1.0则注重性能和简洁性。通过集成AI技术，项目还实现了智能的颜色和谐度评估功能。

未来，该项目可以进一步扩展功能，增加更多分析维度和交互方式，为用户提供更加全面的颜色分析工具。